PROJETO INTERDISCIPLINAR

Requisitos da disciplina Modelagem de Software e Arquitetura de Sistemas

INTEGRANTES DO PROJETO e RA'S

Cauã Ferreira Busto Fernandes - 26028778
Matheus Urquiza Almeida - 26028352

São Paulo

2025

Sumário

1 INTRODUÇÃO 3

1.1 – Processo de desenvolvimento de Software 3

1.2 Apresentação 3

2. DESIGN SPRINT – Ideação e prototipação do desafio 3

2.1 Desafio 3

2.2 Entender Mapear 4

2.3 Ideação – desenho da solução (trilha do usuário) 4

2.4 Prototipagem 4

3.REQUISITOS DE SISTEMA 5

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE 5

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE 7

4. CASOS DE USO 10

5. DIAGRAMA DE CLASSE 10

6. ARQUITETURA DO SISTEMA 10

6.1. Visão Geral do Sistema 10

6.2. Arquitetura Geral 11

6.3. Componentes Arquiteturais e Camadas 11

6.4. Diagrama da Arquitetura 11

6.5. Organização de Pastas e Módulos 12

6.6. Integrações Externas (se houver) 13

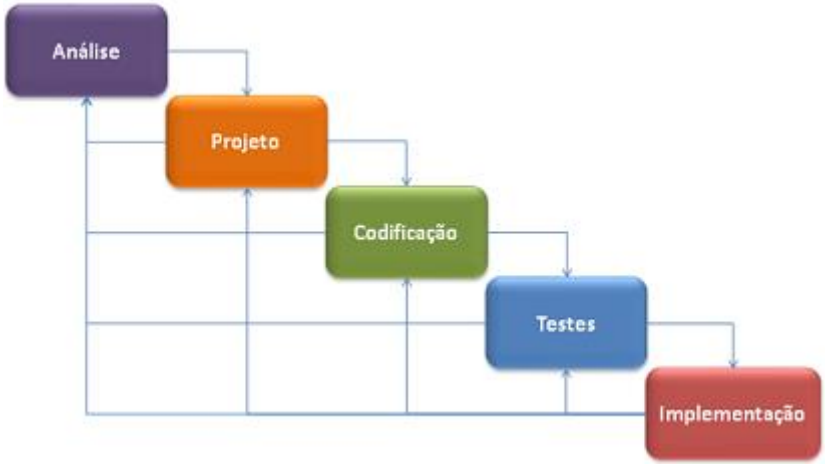
6.7. Ambiente e Implantação 13

8. REFERÊNCIAS BIBLIOGRÁFICAS 13

1 INTRODUÇÃO

1.1 – Processo de desenvolvimento de Software

Exemplo (substituir) - o que foi realizado em sala no Draw.io, escolha um por equipe



1.2 Apresentação

A Messier Data & Creative é uma empresa de desenvolvimento de jogos e experiências interativas que combina tecnologia e criatividade para promover engajamento em eventos, demonstrações de produto e campanhas de marca. Como novo passo, a Messier pretende lançar uma plataforma de jogos digitais para escolas de ensino fundamental e médio. Neste Projeto Interdisciplinar, as equipes irão desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais por escolas, controlando pacotes contratados, limites de acessos mensais, IPs autorizados e relatórios de consumo. Fluxo do processo (simulação do módulo Escola): 1) Login: a escola se autentica; o aplicativo verifica se a escola possui pacote ativo e se o IP de origem (informado na simulação) está autorizado. 2) Catálogo: o aplicativo apresenta os games disponíveis no pacote adquirido pela escola. 3) Acesso: a escola simula o acesso a um game; o aplicativo registra e contabiliza o acesso no consumo do mês, respeitando o limite do pacote.

2. DESIGN SPRINT – Ideação e prototipação do desafio

2.1 Desafio

Desenvolver um aplicativo Desktop para gerenciar assinaturas de jogos educacionais voltados para escolas de ensino fundamental e médio. O sistema deve controlar os pacotes de jogos contratados,

monitorar os limites de acessos mensais, validar os IPs autorizados das instituições e gerar relatórios precisos de consumo, garantindo segurança e organização na disponibilização dos conteúdos.

2.2 Entender Mapear

A etapa de entendimento focou na jornada do usuário (escola) para garantir uma operação sem atritos. O fluxo mapeado consiste em quatro pilares:

Autenticação: A escola realiza o login; o sistema valida automaticamente se há um pacote ativo e se o IP da máquina está na lista de autorizados.

Exploração: O sistema carrega o catálogo exibindo estritamente os jogos pertencentes ao pacote adquirido pela escola.

Consumo: A escola seleciona e acessa um jogo; o sistema registra esse log e desconta o acesso do limite mensal da assinatura.

Gestão: A coordenação da escola acessa a área de relatórios para consultar o volume de consumo da franquia no mês vigente.

2.3 Ideação – desenho da solução (trilha do usuário)

A trilha do usuário foi estruturada em seis etapas sequenciais, focando em usabilidade e processamento em background:

Login: Tela de autenticação com validação de credenciais e IP.

Dashboard: Painel inicial com o resumo do pacote ativo, consumo em tempo real e alertas de limite.

Catálogo de Jogos: Vitrine de jogos disponíveis com opções de filtros por categoria e disciplina.

Acesso ao Jogo: Interface de inicialização rápida da aplicação com controle de sessão.

Registro de Uso: Sistema invisível ao usuário que contabiliza o log, data, hora e atualiza a cota de acessos mensais.

Relatórios: Painel gerencial contendo o histórico detalhado de acessos para prestação de contas e controle da escola.

2.4 Prototipagem

A prototipagem tangibilizou a ideação por meio da criação das telas de maior impacto (Login, Catálogo, Acesso ao Jogo e Relatórios). Os protótipos foram desenvolvidos para validar:

Layout funcional: Interface limpa, organizada e sem poluição visual, adequada ao ambiente educacional.

Navegação intuitiva: Fluxos curtos que exigem o mínimo de cliques para iniciar um jogo.

Validação de regras: Exibição clara de avisos, como o bloqueio de acesso caso o limite mensal do pacote tenha sido atingido ou o IP seja inválido.

3.REQUISITOS DE SISTEMA

3.1 REQUISITOS FUNCIONAIS DE SOFTWARE

RFS01	
Função	Cadastro de Escola
Descrição	Registrar a instituição no sistema para habilitar o acesso.
Entradas	CNPJ, nome da escola e IPs autorizados.
Fonte	Administrador
Saídas	Escola salva no banco de dados
Ação	Salvar os dados no banco.
História de Usuario	Como administrador, quero cadastrar escolas para organizar o acesso e gerenciar as permissões de rede.

RFS02	
Função	Login da Escola
Descrição	Validar as credenciais da instituição para permitir o uso do software.
Entradas	E-mail e senha.
Fonte	Gestor da Escola
Saídas	Acesso liberado ao painel principal
Ação	Validar e-mail e senha.
História de Usuário	Como gestor escolar, quero realizar o login para gerenciar as assinaturas e o catálogo da instituição.

RFFS03	
Função	Criar Pacotes
Descrição	Configurar pacotes de jogos com diferentes limites de acesso.
Entradas	Nome do pacote e limite de acessos mensais.
Fonte	Administrador
Saidas	Pacote disponível para contratação.
Ação	Registrar o plano.
Historia do usuario	Como administrador, quero configurar pacotes de serviços para oferecer diferentes opções de limites e valores.

RSF04	
Função	Filtrar Catálogo
Descrição	Exibir apenas os jogos vinculados ao plano contratado pela escola.
Entradas	Seleção da aba de jogos.
Fonte	Usuário (Gestor ou Aluno)
Saída	Catálogo com títulos liberados
Ação	Carregar a lista.
História de Usuário	Como usuário, quero visualizar apenas os jogos disponíveis no plano para agilizar a escolha da atividade.

RSF05

Função	Consumir Acesso
Descrição	Registrar o uso de um jogo e descontar da cota mensal da escola
Entradas	Inicialização do jogo e IP da máquina.
Fonte	Aluno
Saída	Jogo aberto e log de uso registrado.
Ação	Atualizar saldo de acessos.
História de Usuário	Como aluno, quero iniciar o jogo de forma direta, permitindo o registro automático do consumo da cota.

RSF06	
Função	Relatório de Uso
Descrição	Apresentar o saldo de acessos utilizados pela instituição no mês.
Entradas	Consulta ao painel de relatórios.
Fonte	Gestor da Escola
Saída	Demonstrativo de consumo atualizado.
Ação	Compilar dados de acesso.
História de Usuário	Como gestor, quero acompanhar o relatório de uso para monitorar o saldo de acessos do mês.

3.2 REQUISITOS NÃO FUNCIONAIS DE SOFTWARE

Necessários 6 requisitos

RFS01	
Função	Segurança de Dados
Descrição	Aplicar criptografia para proteção das senhas no banco de dados.
Entradas	Senha definida no cadastro
Fonte	Sistema
Saídas	Hash de senha armazenado
Ação	Criptografar credenciais.
História de Usuário	Como administrador, quero a criptografia de senhas para assegurar a proteção dos dados dos clientes.

RFS02	
Função	Desempenho
Descrição	Garantir tempo de resposta rápido durante a autenticação.
Entradas	Solicitação de login
Fonte	Sistema
Saídas	Carregamento em até 2 segundos
Ação **História de Usuário**	Otimizar tempo de resposta. Como usuário, espero um tempo de resposta ágil para não comprometer o andamento das aulas.

RSF03	
Função	Integridade do Histórico
Descrição	Impedir a alteração ou exclusão dos registros de consumo de jogos. Impedir a alteração ou exclusão dos registros de consumo de jogos.
Entradas	Tentativa de modificação de dados
Fonte	Banco de Dados
Saidas	Saída Bloqueio de alteração.

Ação	Restringir permissões de exclusão.
História de Usuário	Como administrador, quero a imutabilidade dos logs para garantir a transparência e segurança na auditoria.

RSF04	
Função	Persistência
Descrição	Armazenar informações de forma segura contra falhas de hardware
Entradas	Inserção de novos dados.
Fonte	Sistema
Saída	Dados gravados permanentemente.
Ação	Realizar gravação em banco de dados.
História de Usuário	Como gestor, espero a persistência dos dados para evitar perda de informações em falhas de sistema.

RSF05	
Função	Padrão Tecnológico.
Descrição	Desenvolvimento da aplicação utilizando a linguagem C#.
Entradas	Implementação de código.
Fonte	Desenvolvedor
Saída	Software em ambiente .NET.
Ação	Codificar em C#.
História de Usuário	Como desenvolvedor, utilizarei C# para alinhar o projeto às exigências técnicas da instituição

RSF06	
Função	Usabilidade
Descrição	Interface intuitiva para facilitar o uso independente pelo aluno.
Entradas	Interação com o sistema.
Fonte	Interface
Saída	Navegação fluida e simplificada
Ação	Aplicar padrões de UX/UI
História de Usuário	Como usuário, espero uma interface objetiva para facilitar a navegação e o uso independente da ferramenta.

4. CASOS DE USO

Apresentar 3 casos de uso do sistema

5. DIAGRAMA DE CLASSE

6. ARQUITETURA DO SISTEMA

Para a arquitetura do sistema, considere o template abaixo:

Template (esquema de arquitetura) para orientar a elaboração de um **Projeto de Arquitetura de Sistema Web**, cobrindo desde a camada de apresentação até a persistência de dados, com foco em modularidade, escalabilidade e eficiência.

6.1. Visão Geral do Sistema

- **Nome do Sistema**:
- **Objetivo do Sistema**: (Breve descrição da finalidade do sistema)
- **Público-Alvo**:
- **Principais Funcionalidades**:

6.2. Arquitetura Geral

Descrever o estilo arquitetural adotado (por exemplo, MVC, camadas, microsserviços, hexagonal, etc.):

Exemplo: Arquitetura em Camadas (Layered Architecture) com separação clara entre apresentação, lógica de negócios e persistência.

6.3. Componentes Arquiteturais e Camadas

6.3.1. Camada de Apresentação (Frontend)

- **Tecnologias**: (Ex.: HTML5, CSS3, JavaScript, React/Vue/Angular)
- **Responsabilidades**:
  - Interface com o usuário
  - Validação inicial de dados
  - Comunicação com a API via HTTP/JSON

6.3.2. Camada de Aplicação (Backend/API)

- **Tecnologias**: (Ex.: Node.js, Django, Spring Boot, Flask, Laravel)
- **Responsabilidades**:
  - Implementação da lógica de negócio
  - Autenticação/autorização
  - Comunicação com a camada de dados
  - Exposição de endpoints RESTful/GraphQL

6.3.3. Camada de Domínio (Negócio)

- **Descrição**:
  - Contém as regras de negócio e os modelos de domínio
  - Independente das demais camadas
- **Componentes**:
  - Entidades
  - Casos de uso
  - Regras de validação

6.3.4. Camada de Persistência (Banco de Dados)

- **Tecnologias**: (Ex.: PostgreSQL, MySQL, MongoDB, SQLite)
- **Responsabilidades**:
  - Armazenamento e recuperação de dados
  - Scripts de criação e migração de banco

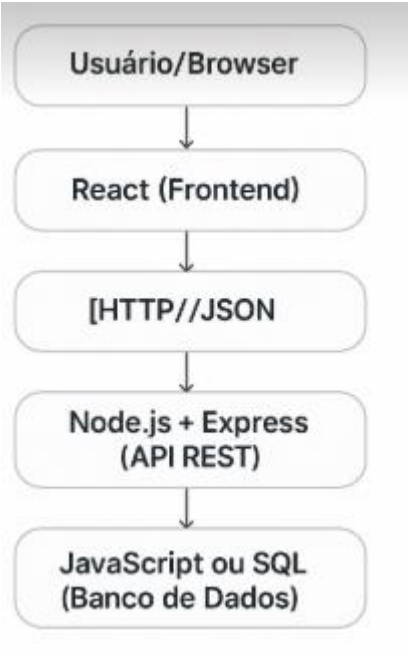
6.4. Diagrama da Arquitetura

Inserir um **diagrama estrutural** (ex.: em draw.io, Lucidchart ou outro) mostrando:

- Fluxo entre frontend, backend, banco de dados
- Componentes principais
- APIs e serviços envolvidos
- Gateways, autenticação, filas de mensagens (se houver)

Exemplo textual de uma arquitetura de sistema web.

[Usuário/Browser]
|
React (Frontend)
|
[HTTP/JSON]
|
Node.js + Express (API REST)
|
JavaScript ou SQL
|
PostgreSQL (Banco de Dados)
Diagrama:



6.5. Organização de Pastas e Módulos

Descrever como o código será organizado no repositório:

```
🗀 /frontend
    🗀 components/
    🗀 pages/
```

```
    🗀 services/
🗀 /backend
    🗀 controllers/
    🗀 services/
    🗀 models/
    🗀 routes/
    🗀 config/
🗀 /database
    🗎 schema.sql
    🗎 migrations/
```

6.6. Integrações Externas (se houver)

- APIs de terceiros (pagamentos, mapas, redes sociais)
- Serviços de autenticação (OAuth2, Firebase, etc.)
- Serviços de envio (email, notificações)

6.7. Ambiente e Implantação

- **Ambientes**: desenvolvimento, homologação, produção
- **Contêineres**: Docker/Docker Compose (se usado)
- **CI/CD**: GitHub Actions, GitLab CI, Jenkins
- **Hospedagem**: Heroku, Vercel, AWS, GCP, Azure

8. REFERÊNCIAS BIBLIOGRÁFICAS

SOMMERVILLE, I. **Engenharia de Software.** 11ª Edição. São Paulo: Pearson Addison-Wesley, 2017.